

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 3

Roll no: 66 PRN No: 12411609 Name of Student: Laukika Shinde

Subject Name: Deep Learning

Problem Statement: Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm:

- 1) Import the required libraries (TensorFlow, Keras, Pandas, NumPy, Matplotlib, and Scikit-learn).
- 2) Upload and load the Pima Indians Diabetes dataset into a Pandas DataFrame.
- 3) Separate the dataset into input features (X) and target variable (y).
- 4) Split the dataset into training (80%) and testing (20%) sets.
- 5) Standardize the feature values using StandardScaler.
- 6) Define a Sequential neural network with two hidden layers and one output layer.
- 7) Compile the model using the Adam optimizer, Binary Crossentropy loss, and Accuracy metric.
- 8) Train the model using forward propagation and backpropagation for different learning rates and epochs.
- 9) Evaluate the trained model on the test dataset.
- 10) Compare the accuracy obtained for different learning rates and epoch values.
- 11) Plot training and validation accuracy and loss graphs.
- 12) Analyze the effect of learning rate and epochs on model performance.

Code:

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from google.colab import files
```

```
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler
```

```
import tensorflow as tf  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense  
from tensorflow.keras.optimizers import Adam
```

```
uploaded = files.upload()  
df = pd.read_csv("diabetes.csv")
```

```
print(df.head())
```

```
X = df.drop("Outcome", axis=1)  
y = df["Outcome"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,  
    random_state=42  
)
```

```
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
def create_model(learning_rate):
```

```
    model = Sequential([  
        Dense(16, activation='relu', input_shape=(X_train.shape[1],)),  
        Dense(8, activation='relu'),  
        Dense(1, activation='sigmoid')  
    ])
```

```
    optimizer = Adam(learning_rate=learning_rate)
```

```
    model.compile(  
        optimizer=optimizer,  
        loss='binary_crossentropy',  
        metrics=['accuracy']  
    )
```

```
    return model
```

```
learning_rates = [0.1, 0.01, 0.001]
```

```
epochs_list = [20, 50, 100]
```

```
results = []
```

```
for lr in learning_rates:

    for epochs in epochs_list:

        print(f"\nTraining Model -> Learning Rate={lr}, Epochs={epochs}")

        model = create_model(lr)

        history = model.fit(

            X_train,

            y_train,

            validation_split=0.2,

            epochs=epochs,

            batch_size=32,

            verbose=0

        )

        loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

        results.append([lr, epochs, accuracy])

        print(f"Test Accuracy: {accuracy*100:.2f}%")

results_df = pd.DataFrame(
```

```
results,

columns=["Learning Rate", "Epochs", "Test Accuracy"]

)

print("\nComparison of Learning Rates and Epochs")

print(results_df)


best_model = create_model(0.001)


history = best_model.fit(

    X_train,

    y_train,

    validation_split=0.2,

    epochs=100,

    batch_size=32,

    verbose=1

)


loss, accuracy = best_model.evaluate(X_test, y_test)


print("\nFinal Test Accuracy: {:.2f}%".format(accuracy*100))


plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
```

```

plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel("Epochs")

plt.ylabel("Accuracy")

plt.title("Training vs Validation Accuracy")

plt.legend()

plt.grid(True)

plt.show()

```

```

plt.figure(figsize=(8,5))

plt.plot(history.history['loss'], label='Training Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel("Epochs")

plt.ylabel("Loss")

plt.title("Training vs Validation Loss")

plt.legend()

plt.grid(True)

plt.show()

```

Output:

Comparison of Learning Rates and Epochs			
	Learning Rate	Epochs	Test Accuracy
0	0.100	20	0.733766
1	0.100	50	0.707792
2	0.100	100	0.668831
3	0.010	20	0.759740
4	0.010	50	0.746753
5	0.010	100	0.720779
6	0.001	20	0.714286
7	0.001	50	0.746753
8	0.001	100	0.701299

Epoch 1/100

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1


```

Training Model -> Learning Rate=0.1, Epochs=20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Test Accuracy: 73.38%

Training Model -> Learning Rate=0.1, Epochs=50
Test Accuracy: 70.78%
Test Accuracy: 70.78%

Training Model -> Learning Rate=0.1, Epochs=100
Test Accuracy: 66.88%

Training Model -> Learning Rate=0.01, Epochs=20
Test Accuracy: 75.97%

Training Model -> Learning Rate=0.01, Epochs=50
Test Accuracy: 74.68%

Training Model -> Learning Rate=0.01, Epochs=100
Test Accuracy: 72.08%

Training Model -> Learning Rate=0.001, Epochs=20
Test Accuracy: 71.43%

Training Model -> Learning Rate=0.001, Epochs=50
Test Accuracy: 74.68%

Training Model -> Learning Rate=0.001, Epochs=100
Test Accuracy: 70.13%

```

Final Test Accuracy: 75.32%



